

AIMETEO: Specifica Tecnica Backend AI (v3.0)

Roadmap di implementazione Deep-Dive per il triennio 2026-2028

Questo documento dettaglia l'architettura tecnica necessaria per trasformare **AIMETEO** da un'interfaccia prototipale a un ecosistema di sicurezza alpina guidato dall'intelligenza artificiale. L'obiettivo è la creazione di un motore decisionale che separi nettamente il *dato deterministico* (meteo e bollettini) dalla *sintesi linguistica* (AI).

Fase 1: Ingestione Multi-Sorgente e Normalizzazione

Non possiamo dipendere da una sola API. Il backend deve diventare un aggregatore resiliente.

1.1 Pipeline Meteo (ETL)

- **Sorgenti Primarie:** Integrazione via Python delle API di *Google Maps Weather* (per la precisione locale) e *ECMWF (Open-Meteo)* come fallback.
- **Dati GRIB2:** Implementazione di librerie come *cf_grib* e *xarray* per elaborare i file meteorologici grezzi e mappare i campi di pressione, vento a 3000m e zero termico.
- **Frequenza:** Pooling ogni 60 minuti con salvataggio su *GCP BigQuery* per analisi storica (fondamentale per addestrare modelli futuri sulle micro-previsioni locali).

1.2 Scraper Bollettini Valanghe (AINEVA/Meteomont)

- **Automazione:** Scripting con Selenium/Playwright per estrarre i dati dai siti regionali che non offrono API (es. ARPA Lombardia, Veneto, Piemonte).
- **Parser Strutturato:** Conversione del testo libero ("Pericolo marcato sopra i 2000m su esposizioni Nord") in vettori numerici utilizzabili dal codice di sicurezza.

Python 3.12

GCP Cloud Functions

BeautifulSoup

Fase 2: Il "Cervello Determinista" (Route DB & Safety Engine)

L'AI non calcola il rischio; lo spiega. Il calcolo avviene qui.

2.1 Architettura del Route Database

Ogni itinerario nel database deve contenere metadati granulari:

- **Geometria:** Tracciato GPX con campionamento pendenza ogni 50 metri.
- **Esposizione:** Mappatura polare delle pareti (N, NE, E, etc.) per incrocio con l'irraggiamento solare previsto.

PROPRIETÀ RISERVATA AIMETEO SRL - DOCUMENTAZIONE TECNICA 2026 - CONFIDENZIALE

- **Metadati Tecnici:** Quota di attacco, quota massima, difficoltà (UIAA/grado sciistico), tempi medi di percorrenza.

2.2 Il Safety Filter (Algebra della Montagna)

Un modulo Python che esegue test booleani prima di ogni risposta:

```
IF (snow_alert AND route.slope > 30) OR (wind_speed > 50kmh) OR (temp_rise > 5deg AND route.exposure == 'S'):  
    STATUS = "DANGEROUS"  
    ACTION = "BLOCK_RECOMMENDATION"
```

Questo garantisce che Gemini non suggerisca mai un percorso tecnicamente a rischio basandosi su allucinazioni probabilistiche.

PostgreSQL + PostGIS

FastAPI

Pydantic

Fase 3: Integrazione Vertex AI e Prompt Engineering

Trasformiamo i dati in una "Relazione di Guida Alpina".

3.1 Architettura RAG (Retrieval-Augmented Generation)

Invece di mandare tutto a Gemini, il backend seleziona i 3 itinerari migliori e invia al modello solo i dati necessari:

- **System Instruction:** "Agisci come una Guida Alpina UIAGM. Usa un tono asciutto, tecnico e prudente. Cita sempre la fonte ufficiale del bollettino valanghe."
- **Function Calling:** Obblighiamo il modello a restituire un JSON che contenga: titolo, descrizione_condizioni, equipaggiamento_consigliato, allerta_sicurezza.

3.2 Gestione Hallucinations

- **Grounding:** Utilizziamo il servizio di Grounding di Vertex AI per forzare il modello a citare esclusivamente i dati meteo forniti nel contesto della richiesta.

Google Gemini 2.x Flash

Vertex AI SDK

LangChain

Fase 4: Infrastructure, Scalability & Legal Compliance

Preparare l'app per 100.000+ utenti e gestire le responsabilità civili.

4.1 Performance e Caching

- **Global Edge Caching:** Le previsioni per zone specifiche (es. "Massiccio del Sella") vengono generate una volta ogni 2 ore e salvate in *GCP Memorystore (Redis)*. Questo riduce i costi API del 90%.
- **Cloud Run:** Deploy dell'app in container Docker su Cloud Run per gestire picchi di traffico (es. Venerdì sera prima del weekend).

4.2 La "Scatola Nera" (Liability Audit Log)

In caso di incidente in montagna, l'azienda deve poter dimostrare cosa ha mostrato all'utente. Implementiamo un sistema di logging immutabile su *Cloud Logging* che salva per 10 anni:

- Prompt inviato all'AI.
- Dati meteo grezzi associati.
- Bollettino valanghe ufficiale in vigore in quel momento.
- Risposta finale generata.

Docker

GCP Cloud Run

Redis

Terraform

PUNTO CRITICO DI BUSINESS: Il valore di AIMETEO non risiede nel codice dell'app, ma nella qualità del Route DB e nella precisione dei Safety Filters. Lo sviluppo tecnico deve dare priorità assoluta alla pulizia del database percorsi rispetto all'estetica del frontend.